

render method. This render method returns the content to be displayed on the web page.

You may ask the question as to why we go to the effort of creating a class when we can use the simple function (as shown in the first part of the code) to achieve the same results. That's because, until React version 16.8, we could use functional components only if we planned on using props and never state. So if you want to use state, we need to compulsorily create a class, as seen in the stateful component.

```
class MyApp extends React.Component {
  constructor () {
    super()
    this.state = {
      name: ''
    }
  }

  render() {
    <SayHello name={this.state.name} />
  }
}
```

In the above code, there's a stateful component, where *name* is a string in the state. We can access its value using *this.state.name* and we can set the state value using *this.setState*.

Component life cycle

Let's take a look at the component life cycle. Life cycle methods are invoked at different phases of the life cycle of a component. These methods are provided so that the developer can produce a quality application, and really plan what and how to do this at various stages of the UI interface. There are four phases of any component's life cycle, the first one being

'initialisation'. In this phase, the React component prepares by setting up the initial state and the default props. The initial state is set in the constructor as we saw in the previous example code, and can later be changed using the *setState* method. The default props are defined as a property of the component to define all the default values, which can be overridden by the new prop values as we want in the component.

The next phase is 'mounting'. Now that we have prepared the state and props, the React component is ready to mount in the browser DOM. *componentWillMount* is executed just before the React component is about to be mounted. Hence, all the things that you want to do before a component mounts have to be defined here. This method is executed once in the life cycle of a component and before the first render. The next method under the mounting phase is *render*. Simply put, it basically mounts the component onto the browser. The *componentDidMount* is executed once in the life cycle of a component and after the first render. Here we can access the DOM, so we should initialise the JavaScript libraries that need DOM access. Also, all the API calls should be made in the *componentDidMount* always, because that's where we can access all the DOM elements.

The next phase is called 'updating'. It starts when the React component is in the browser. So let's first look at the methods that get called when the parent sends new props to the child. *componentWillReceiveProps* gets executed when the props have changed, and this is not the first render. This is the method where we can update the state with the values of the new props, i.e., syncing the state with the props. *shouldComponentUpdate* tells React whether to re-render when the component receives new props. As you can see, this method's name is like a question: should the component update? So it returns *true* or *false*; and if it returns *true*, the component updates. *componentWillUpdate* is only used to prepare for the upcoming render. Similar to

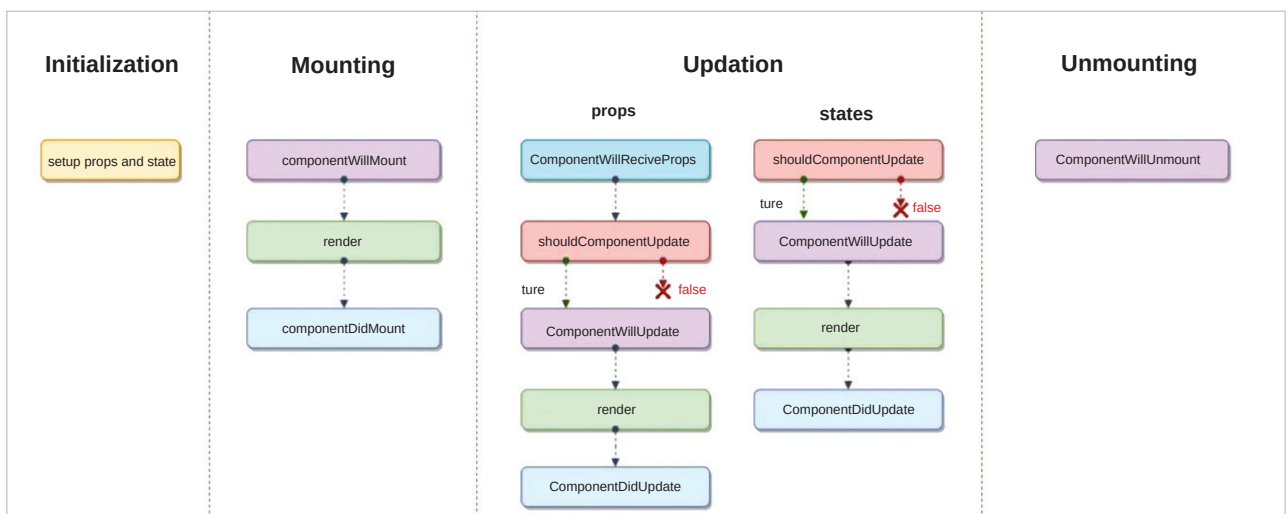


Figure 2: Component life cycle